AI Kernel Architecture Specification

Project NOVA Central Orchestration Engine, Lifecycle Coordination, and Core Interface Abstractions

---

Field	Value
Document ID	NOVA-SPEC-009
Version	1.0
Status	`APPROVED`
Author	Antigravity (Lead Software Engineering Agent)
Reviewer	ChatGPT (Chief Architect)
Approved By	Praveen (Project Founder)
Created	2026-06-28
Last Updated	2026-06-28
Dependencies	NOVA-SPEC-001, NOVA-SPEC-002, NOVA-SPEC-003, NOVA-SPEC-004

Revision History

Version	Date	Author	Summary of Changes
1.0	2026-06-28	Antigravity	Initial publication of the formal AI Kernel Architecture Specification.

Table of Contents

Purpose & Scope

This specification defines the system architecture, module boundaries, lifecycle sequences, state machine, and interface contracts for the **AI Kernel** of Project NOVA. The AI Kernel serves as the central coordination and execution orchestration subsystem, binding all stateful intelligence and stateless automation capabilities together.

Core Responsibilities & Boundaries

1. Primary Orchestrator Role

The AI Kernel operates exclusively as the cognitive coordinator of Project NOVA. It directs data flows, handles execution states, audits task permissions, and coordinates failure recovery.

2. Isolation of Concerns (The Coordination Mandate)

The AI Kernel **MUST NOT** directly implement local capabilities, driver logic, or third-party API processing.

- **Voice/STT/TTS:** Handled exclusively by the Voice Capability (`VoiceEngine`).
- **DPI Screen Observation/OCR:** Handled exclusively by the Vision Capability (`VisionEngine`).
- **Semantic Context Persistent Lookup:** Handled by the Memory Capability (`MemoryEngine`).
- **Natural Intention Decompositions:** Handled by the Planning Capability (`PlannerEngine`).
- **OS Automation (Clicks, Window movements, Key inputs):** Handled by Desktop and Browser Capability wrappers.

- **Validation Rules:** Handled by the Permissions Capability.

Internal Component Model

The internal modules of the AI Kernel cooperate to maintain execution session contexts:

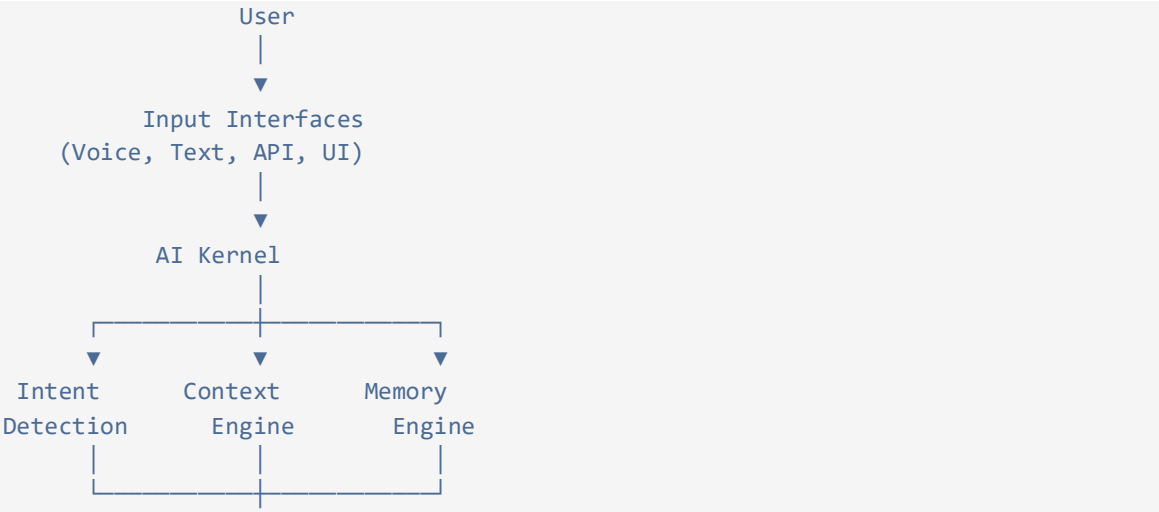
- **Orchestrator Core:** The central coordination controller that sets up session loops, registers active engines, and routes events.
- **EventBus:** A thread-safe message broker implementing the Publisher/Subscriber pattern to handle inter-subsystem updates.
- **StateManager:** Tracks execution status and enforces valid state transitions.
- **ToolRegistry:** Handles tool registrations, matching planner tasks to tool descriptors, and invoking validation paths.
- **ExceptionRouter:** Catches, logs, and routes runtime errors, managing loop recovery options.

```
graph TD
    subgraph AI_Kernel["AI Kernel (Specification Boundaries)"]
        OC[Orchestrator Core] <--> SM[StateManager]
        OC <--> EB[EventBus]
        OC <--> TR[ToolRegistry]
        OC <--> ER[ExceptionRouter]
    end
    EB <--> Subsystems["External Subsystems (Voice, Vision, Planner, Memory)"]
```

Execution Lifecycle

The execution lifecycle of a single user transaction is governed by the central AI Kernel. The global data and coordination flow is structured as follows:

Global Transaction Flow Diagram



```
                  |
                  ▼
          Planner Engine
                  |
                  ▼
         Permission Engine
                  |
                  ▼
          Tool Selection
                  |
                  ▼
        Execution Engine
                  |
                  ▼
       Observation Engine
                  |
                  ▼
        Reflection Engine
                  |
                  ▼
          Memory Update
                  |
                  ▼
        Response Generator
                  |
                  ▼
                User
```

Visual Transaction Model

```
graph TD
    UserNode([User]) --> Input[Input Interfaces<br/>Voice, Text, API, UI]
    Input --> Kernel[AI Kernel]

    Kernel --> Intent[Intent Detection]
    Kernel --> Context[Context Engine]
    Kernel --> Memory[Memory Engine]

    Intent --> Planner[Planner Engine]
    Context --> Planner
    Memory --> Planner

    Planner --> Permissions[Permission Engine]
    Permissions --> Selection[Tool Selection]
    Selection --> Execution[Execution Engine]
    Execution --> Observation[Observation Engine]
    Observation --> Reflection[Reflection Engine]
    Reflection --> MemoryUpdate[Memory Update]
    MemoryUpdate --> Response[Response Generator]
    Response --> UserNode
```

Lifecycle Stage Details

The transaction lifecycle aggregates these stages:

12. **Session Setup:** Orchestrator initializes, registers subsystems, and starts thread queues.
13. **Listening/Triggering:** Microphone listeners wait for wake triggers, or text input triggers are received.
14. **Intent Parsing:** Audio frames are transcribed to text, and intent classifiers analyze intent structures.
15. **Plan Compilation:** The context aggregation engine builds system snapshots, and the Planner compiles a task list.
16. **Permissions Verification:** The Tool Manager validates each plan task against security policy rules.
17. **Action Dispatch:** The Execution Engine converts tasks to action primitives and dispatches them to OS wrappers.
18. **Outcome Verification:** The Observation Engine checks screen updates to verify action outcomes.
19. **Session Shutdown:** The loop closes, active handles release, and memory updates persist to storage tables.

State Machine & Transitions

The active lifecycle state of the AI Kernel is governed by a strict state machine:

```
stateDiagram-v2
    [*] --> Idle
    Idle --> Initializing : StartSession
    Initializing --> Listening : InitComplete
    Listening --> Parsing : InputReceived
    Parsing --> Planning : IntentDetected
    Planning --> Verifying : PlanGenerated
    Verifying --> Executing : PlanApproved
    Verifying --> Planning : PermissionDenied
    Executing --> Reflecting : ActionCompleted
    Reflecting --> Executing : RetryRequired
    Reflecting --> Listening : GoalAchieved
    Reflecting --> Error : ExecutionFailed
    Error --> Idle : ResetSession
    Paused --> Executing : Resume
    Executing --> Paused : Suspend
    Listening --> Terminating : StopSession
    Terminating --> [*]
```

Transition Transition Matrix

Current State	Input Event	Target State	Description

Idle	`StartSession`	**Initializing**	Sets up active threads and listeners.
Initializing	`InitComplete`	**Listening**	Begins active microphone or prompt polling.
Listening	`InputReceived`	**Parsing**	Converts audio/text to intent payloads.
Parsing	`IntentDetected`	**Planning**	Requests task decompositions.
Planning	`PlanGenerated`	**Verifying**	Submits plans to permission checks.
Verifying	`PlanApproved`	**Executing**	Runs task adapters.
Verifying	`PermissionDenied`	**Planning**	Requests alternate plans from the Planner.
Executing	`ActionCompleted`	**Reflecting**	Verifies outcomes via screen observation.
Reflecting	`RetryRequired`	**Executing**	Re-executes task with updated bounds.
Reflecting	`GoalAchieved`	**Listening**	Awaits next user goal.
Reflecting	`ExecutionFailed`	**Error**	Initiates diagnostic logs and halts loop.

Public Interface Definitions

The AI Kernel implements these core API structures:

```
from abc import ABC, abstractmethod
from typing import Callable, Any, Generator

class IEventBus(ABC):
    @abstractmethod
    def publish(self, topic: str, event_data: dict) -> None: ...
    @abstractmethod
    def subscribe(self, topic: str, handler: Callable[[dict], None]) -> None: ...
    @abstractmethod
    def unsubscribe(self, topic: str, handler: Callable[[dict], None]) -> None:
...
```

```
class IStateManager(ABC):
    @abstractmethod
    def get_current_state(self) -> str: ...
    @abstractmethod
    def transition_to(self, target_state: str) -> bool: ...
    @abstractmethod
    def register_transition_hook(self, callback: Callable[[str, str], None]) ->
None: ...

class IAIKernel(ABC):
    @abstractmethod
    def initialize(self) -> None: ...
    @abstractmethod
    def process_goal(self, goal_text: str) -> Generator[dict, None, None]: ...
    # Yields status telemetry frames: {"status": "planning", "step": 1}
    @abstractmethod
    def suspend(self) -> None: ...
    @abstractmethod
    def resume(self) -> None: ...
    @abstractmethod
    def shutdown(self) -> None: ...

class IExceptionRouter(ABC):
    @abstractmethod
    def route_error(self, error: Exception, context_data: dict) -> str: ...
    # Returns recovery status: "RETRY", "ABORT", or "REQUIRES_USER_INTERVENTION"
```

Subsystem Dependency Graph

The AI Kernel sits at the cognitive layer, depending strictly on interfaces, never concrete execution wrappers:

```
graph TD
    AI_Kernel[AI Kernel] --> IEventBus[IEventBus Interface]
    AI_Kernel --> IStateManager[IStateManager Interface]
    AI_Kernel --> IPlanner[IPlanner Interface]
    AI_Kernel --> IMemoryEngine[IMemoryEngine Interface]
    AI_Kernel --> IToolManager[IToolManager Interface]

    IToolManager --> IDesktopCapability[IDesktopCapability]
    IToolManager --> IBrowserCapability[IBrowserCapability]

    classDef interface fill:#e1f5fe,stroke:#01579b,stroke-width:2px;
    class
IEventBus,IStateManager,IPlanner,IMemoryEngine,IToolManager,IDesktopCapability,IBr
owserCapability interface;
```

Communication & Messaging Model

- **Asynchronous Message Loop:** Subsystems write events to the `IEventBus`. Event distribution runs in a thread-safe asynchronous loop.
- **Topic Schemas:** Subsystems publish to strict topic categories:
- `system/lifecycle/*` — State transitions and heartbeats.
- `user/input/*` — Audio captures, transcribed text.
- `planning/plan/*` — Task compiling, permission statuses.
- `execution/actions/*` — Mouse emulations, terminal scripts.
- `observation/frames/*` — Screenshots, parsed DOM layout outputs.

Error Propagation & Recovery Strategy

To satisfy "Security by Design" (`NOVA-002`), failures must be managed systematically:

1. The Abort Standard

If any action fails to verify after three retry loops, or if a permission violation is logged, the `ExceptionRouter` forces a `SUSPEND` state. This triggers an immediate execution abort and stops keyboard/mouse events.

2. Reflection Retries

When a tool execution fails with a recoverable exit code (e.g. Git repository lock error), the exception is logged to `Reflection`. The Planner receives the failure context and compiles an alternative plan.

3. Critical Crash Recovery

If a subsystem thread crashes, the Orchestrator catches the unhandled exception, publishes a `SystemCrashedEvent`, releases hardware device contexts, and shifts the state machine to `ERROR` to safeguard workspace files.

Logging, Observability, and Observability Metrics

A. Trace ID Tracing

Every intent cycle receives a unique `Trace ID`. Subsystems must prepend this string to all published events and log entries to enable end-to-end debugging.

B. Obfuscation Policies

To protect user privacy, parameters marked as sensitive (e.g. inputs containing keys, passwords, or personal details) must be obfuscated in log outputs:

```
# Raw input:
{"action": "type", "selector": "#password", "content": "RaviPass123"}
# Obfuscated log output:
{"action": "type", "selector": "#password", "content": "[REDACTED]"}
```

C. Telemetry Metrics

The StateManager exposes metric status registers:

- `transaction_latency_ms` — Processing latency from user query to action execution.
- `active_state` — Current kernel machine state.
- `failed_executions_count` — Count of execution retries.

Extensibility & Plugin Points

The AI Kernel supports third-party tool integrations through the `ToolRegistry`:

- **Plugin Registration:** External modules expose a descriptor containing tool functions, descriptions, expected parameters, and permission levels.
- **Dynamic Matching:** During the planning phase, the Planner queries the registry to match intent structures against registered plugin tools.
- **Security Sandboxing:** Plugins must register via abstract APIs and run within the validation checks of the Tool Manager. Direct system hooks are restricted.